UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 3

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : Sazid Abdullah

STUDENT ID : 23101146

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 16 - 08 - 2026

SUBMITTED TO

AITA RAHMAN ORTHI

LECTURER AT UAP

1. Problem:

Implement the following **CPU Scheduling Algorithms** for the given set of processes:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	7
P2	1	4
P3	2	15
P4	3	11
P5	4	20
P6	4	9

Implement the following scheduling algorithms:

- **Round Robin (RR)** with **Time Quantum = 5**
- **Shortest Job First (SJF) – Non-Preemptive**
- **First Come First Serve (FCFS)**

For each scheduling algorithm, calculate:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**
- **Average Turnaround Time (AVG TAT)**
- **Average Waiting Time (AVG WT)**

Also, represent the **process execution sequence/Gantt Chart** for each algorithm.

Finally, **compare the Average TAT and Average WT** of Round Robin, SJF, and FCFS and determine which scheduling algorithm performs better for the given set of processes. Explain the reason for the difference in performance.

2. Source Code:

```
processes = [
    ["P1", 0, 7],
    ["P2", 1, 4],
    ["P3", 2, 15],
    ["P4", 3, 11],
    ["P5", 4, 20],
    ["P6", 4, 9],
]



process = [{"pid": p[0], "at": p[1], "bt": p[2]} for p in processes]

n = len(process)
completed = []
time = 0
done = 0
is_done = [False] * n

while done < n:
    ready = [p for i, p in enumerate(process) if p["at"] <= time and not is_done[i]]
    if not ready:
        time = min(p["at"] for i, p in enumerate(process) if not is_done[i])
        continue

    current = min(ready, key=lambda p: p["bt"])
    idx = process.index(current)

    start = max(time, current["at"])
    finish = start + current["bt"]
    tat = finish - current["at"]
    wt = tat - current["bt"]

    completed.append({**current, "ct": finish, "tat": tat, "wt": wt})
```

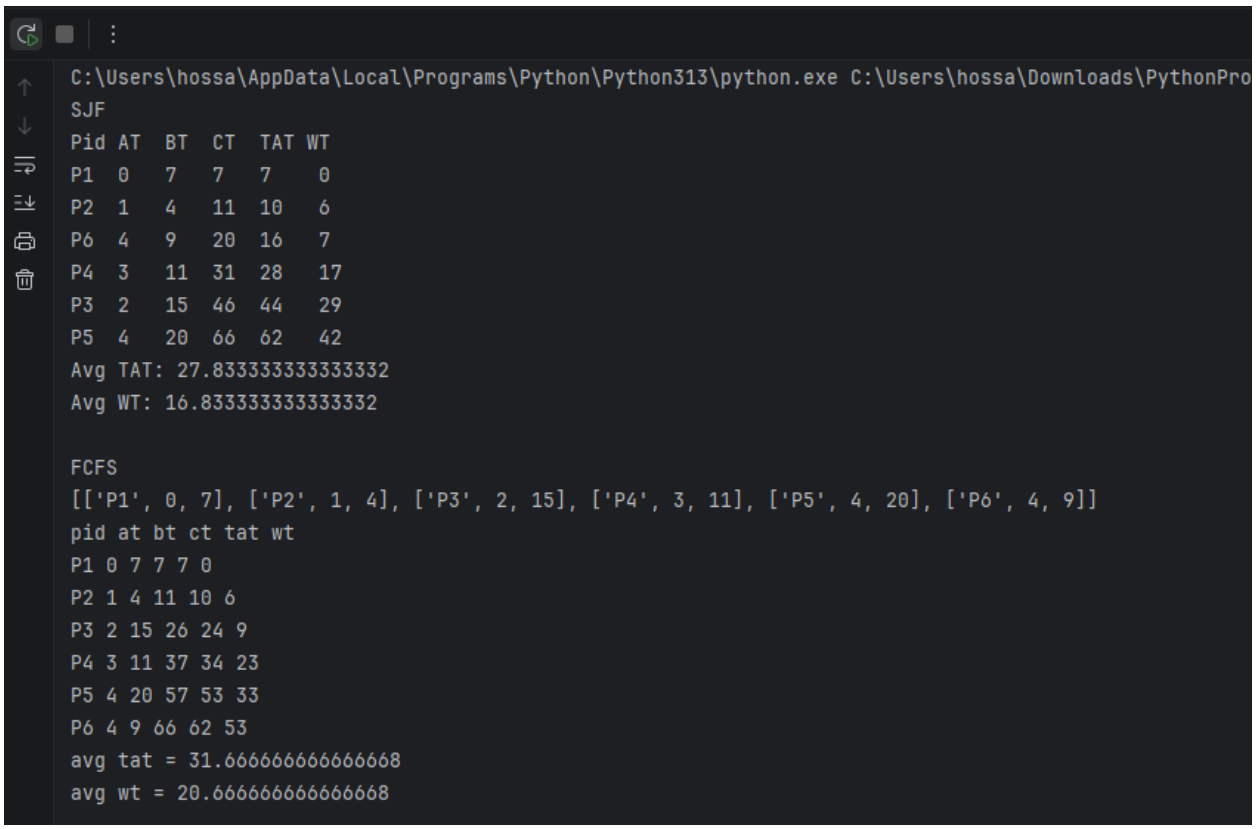
```
    is_done[idx] = True
    time = finish
    done += 1

print("SJF")
print(f"{'Pid':<4}{'AT':<4}{'BT':<4}{'CT':<4}{'TAT':<4}{'WT':<4}")
for p in completed:
    print(f"{p['pid']:<4}{p['at']:<4}{p['bt']:<4}{p['ct']:<4}{p['tat']:<5}{p['wt']:<4}")

sjf_avg_tat = sum(p["tat"] for p in completed) / n
sjf_avg_wt = sum(p["wt"] for p in completed) / n

print(f"Avg TAT: {sjf_avg_tat}")
print(f"Avg WT: {sjf_avg_wt}")



print()
print("FCFS")

lst = sorted(processes, key=lambda x: x[1])   # AT onujayi sort
print(lst)

ct = []
tat = []
wt = []
time = 0

for p in lst:
    pid, at, bt = p[0], p[1], p[2]

    if time < at:
        time = at
    time += bt

    completion = time
    turnaround = completion - at
```

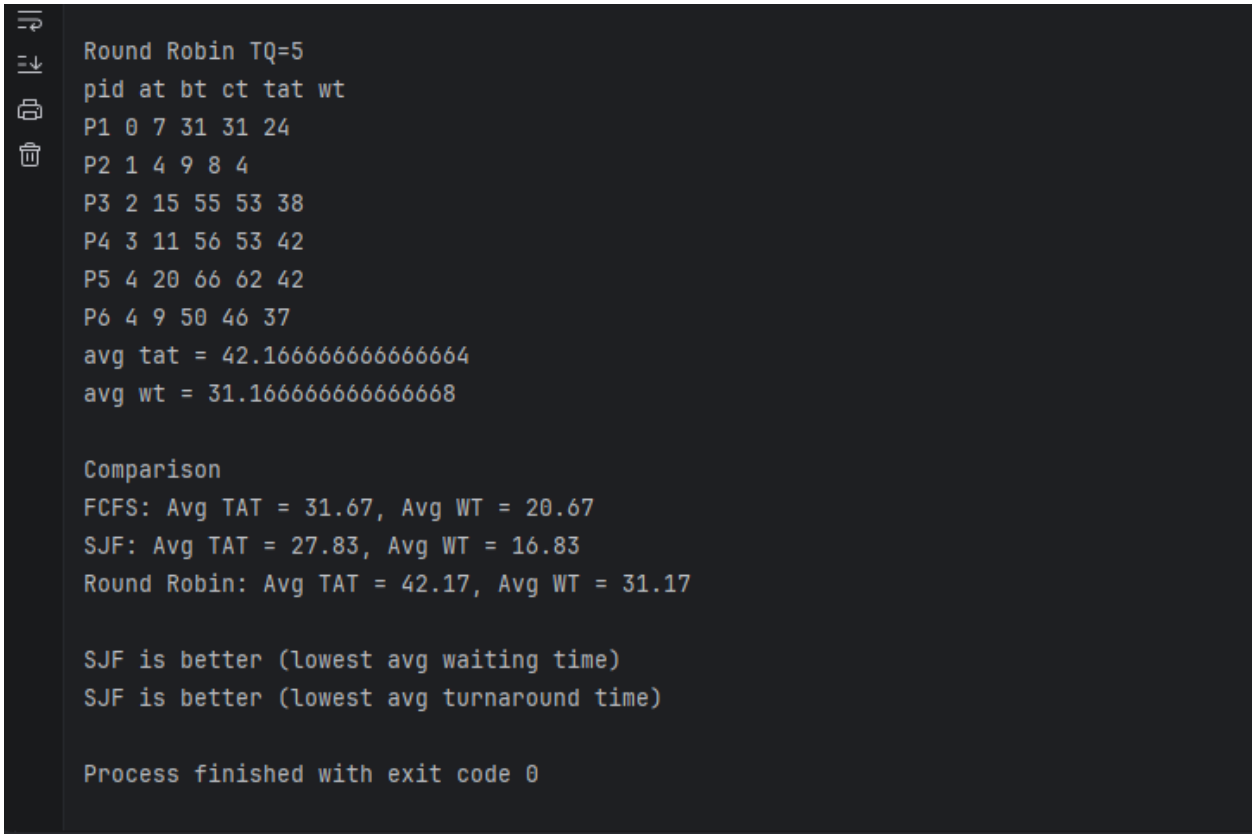
```
    waiting = turnaround - bt

    ct.append(completion)
    tat.append(turnaround)
    wt.append(waiting)

print("pid", "at", "bt", "ct", "tat", "wt")
for i in range(len(lst)):
    print(lst[i][0], lst[i][1], lst[i][2], ct[i], tat[i], wt[i])

fcfs_avg_tat = sum(tat) / len(lst)
fcfs_avg_wt = sum(wt) / len(lst)

print("avg tat =", fcfs_avg_tat)
print("avg wt =", fcfs_avg_wt)



print()
print("Round Robin TQ=5")

tq = 5
rr_lst = sorted(processes, key=lambda x: x[1])

remaining_bt = {p[0]: p[2] for p in rr_lst}
at_map = {p[0]: p[1] for p in rr_lst}
bt_map = {p[0]: p[2] for p in rr_lst}

queue = []
time = 0
i = 0
ct_rr = {}

queue.append(rr_lst[0][0])
i = 1

while queue:
```

```
    pid = queue.pop(0)
    run = min(tq, remaining_bt[pid])
    time += run
    remaining_bt[pid] -= run


    while i < len(rr_lst) and rr_lst[i][1] <= time:
        queue.append(rr_lst[i][0])
        i += 1

    if remaining_bt[pid] > 0:
        queue.append(pid)
    else:
        ct_rr[pid] = time

rr_tat = []
rr_wt = []

print("pid", "at", "bt", "ct", "tat", "wt")
for p in rr_lst:
    pid, at, bt = p[0], p[1], p[2]
    turnaround = ct_rr[pid] - at
    waiting = turnaround - bt
    rr_tat.append(turnaround)
    rr_wt.append(waiting)
    print(pid, at, bt, ct_rr[pid], turnaround, waiting)

rr_avg_tat = sum(rr_tat) / len(rr_lst)
rr_avg_wt = sum(rr_wt) / len(rr_lst)

print("avg tat =", rr_avg_tat)
print("avg wt =", rr_avg_wt)



print()
print("Comparison")
```

```
results = {
    "FCFS": (fcfs_avg_tat, fcfs_avg_wt),
    "SJF": (sjf_avg_tat, sjf_avg_wt),
    "Round Robin": (rr_avg_tat, rr_avg_wt),
}

for name, (avg_tat, avg_wt) in results.items():
    print(f"{name}: Avg TAT = {avg_tat:.2f}, Avg WT = {avg_wt:.2f}")

best_wt = min(results, key=lambda k: results[k][1])
best_tat = min(results, key=lambda k: results[k][0])

print(f"\n{best_wt} is better (lowest avg waiting time)")
print(f"{best_tat} is better (lowest avg turnaround time)")
```

3. Output:


C:\Users\hossa\AppData\Local\Programs\Python\Python313\python.exe C:\Users\hossa\Downloads\PythonPro
SJF
Pid AT BT CT TAT WT
P1 0 7 7 7 0
P2 1 4 11 10 6
P6 4 9 20 16 7
P4 3 11 31 28 17
P3 2 15 46 44 29
P5 4 20 66 62 42
Avg TAT: 27.833333333333332
Avg WT: 16.833333333333332
FCFS
[['P1', 0, 7], ['P2', 1, 4], ['P3', 2, 15], ['P4', 3, 11], ['P5', 4, 20], ['P6', 4, 9]]
pid at bt ct tat wt
P1 0 7 7 7 0
P2 1 4 11 10 6
P3 2 15 26 24 9
P4 3 11 37 34 23
P5 4 20 57 53 33
P6 4 9 66 62 53
avg tat = 31.666666666666668
avg wt = 20.666666666666668



4. Discussion:

This experiment evaluated three CPU scheduling techniques: Round Robin (RR), Shortest Job First (SJF), and First Come First Serve (FCFS), using an identical set of six processes. Their performances were analyzed by comparing Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

Among the three methods, SJF produced the most efficient results for the given workload. It achieved an Average TAT of **27.83 units** and an Average WT of **16.83 units**, both of which were the lowest. FCFS showed comparatively moderate results, with an Average TAT of **31.67 units** and an Average WT of **20.67 units**. On the other hand, Round Robin, using a time quantum of 5 units, resulted in an Average TAT of **42.17 units** and an Average WT of **31.17 units**.

The lower waiting and turnaround times of SJF indicate that selecting shorter processes first can improve CPU scheduling efficiency for this particular process set. FCFS is simpler because processes are handled according to their arrival order, but longer processes can cause other processes to wait. RR ensures that each process receives regular CPU time, making it suitable for interactive and time-sharing environments; however, the repeated context switching and time-slice allocation led to higher waiting and turnaround times in this experiment.

5. Conclusion:

The experiment demonstrated the performance differences between RR, SJF, and FCFS scheduling algorithms. Based on the calculated CT, TAT, and WT values, **SJF provided the best performance for the selected processes**, achieving the lowest Average TAT (**27.83**) and Average WT (**16.83**).
FCFS ranked second, with an Average TAT of **31.67** and an Average WT of **20.67**, while RR produced the highest values, with an Average TAT of **42.17** and an Average WT of **31.17**.
Although RR has the advantage of providing fair CPU time to every process, it was not the most efficient method for this particular workload. Therefore, when the primary objective is to reduce waiting and turnaround time for these processes, **SJF is the most suitable scheduling algorithm among the three tested methods**.

6. GitHub:

https://github.com/AbirOPS2/CSE402-Operating-System-Lab/tree/main/Lab-04-RR-Comparison-with-FCFS-SJF